BETTER, TOGETHER

Therapy Continuity Companion

Project Overview • Architecture • Setup • Data Flow • Deployment

Purpose

A web application concept designed to support continuity between therapy sessions through patient check-ins and therapist preparation. Patients record structured wellbeing responses and optional notes; therapists can review saved check-ins and prepare for a session using information available in the app.

Current implementation

This guide documents the MVP as described during development. Its follow-up prompts and session-preparation brief are rule-based/local logic, not a connected AI model. The application is a support tool, not a diagnostic system or a replacement for professional care.

Repository companion document

1. Project at a glance

Better, Together (Therapy Continuity Companion) is a Next.js web application using TypeScript and Supabase. It supports a patient check-in flow and a therapist dashboard that reads stored check-in information.

Area	Current project details
Frontend / framework	Next.js App Router, React, TypeScript
Backend / data	Supabase client and PostgreSQL tables
Authentication	Supabase Auth; profiles are populated from signup metadata via a database trigger
Patient experience	Ten multiple-choice questions, optional text, contextual follow-up, saved responses and wellbeing scores
Therapist experience	Dashboard showing check-ins, follow-up responses and a session-preparation brief
Deployment target	Vercel connected to the GitHub repository

2. Main user journeys

Patient journey

• The patient signs up or logs in through Supabase authentication.

• The patient opens the check-in page and answers ten multiple-choice questions.

• The app derives a contextual follow-up question from the responses and allows an optional text answer.

• The check-in is saved to Supabase, including scores, answers and rule-based follow-up fields.

• Saved records remain in the database subject to access policies.

Therapist journey

• The therapist signs in and opens the therapist dashboard.

• The dashboard reads check-ins for patients associated with that therapist.

• The therapist reviews check-in responses and the follow-up answer.

• The Prepare for session action builds a local/rule-based preparation brief from the saved record.

3. Technology and repository layout

The project is a Next.js App Router application. These paths reflect files observed in the repository commit output.

```
app/
layout.tsx
page.tsx
login/page.tsx
signup/page.tsx
patient/checkin/page.tsx
therapist/dashboard/page.tsx
lib/
supabase.ts
public/
(static assets)
package.json
package-lock.json
next.config.ts
tsconfig.json
eslint.config.mjs
```

The Supabase client is initialized in lib/supabase.ts using createClient from @supabase/supabase-js. Environment variables are read from the runtime environment rather than hard-coded in source.

Dependencies and tools

• Next.js App Router and React with TypeScript.

• Supabase JavaScript client (@supabase/supabase-js).

• Node.js/npm for local development and builds.

• Git and GitHub for source control.

• Vercel as the hosting platform.

4. Supabase data model

The following table names and fields are based on implementation details recorded during development. Verify the live Supabase schema if exact SQL definitions are needed.

Table	Purpose / known fields
profiles	id (references auth.users), full_name, role (patient or therapist)
therapist_patients	Links therapist_id to patient_id
checkins	patient_id, date, mood_score, stress_score, energy_score, patient_note, ai_followup_question, ai_followup_answer, ai_structured_summary, ai_topics, questionnaire_answers (JSONB), created_at; likely an id field

A Supabase Auth trigger populates profiles from signup metadata, including name and role. A therapist_patients relationship associates patients with therapists. Row Level Security (RLS) policies should restrict access according to the intended patient/therapist relationship.

Security note: verify every table's RLS policies and test with separate patient and therapist accounts before using real or sensitive information. A dashboard filter alone is not access control.

5. Environment configuration

Create a local .env.local file in the project root. Keep it out of Git.

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=your_supabase_publishable_key
```

Use the exact values from your Supabase project. Never expose a Supabase service-role/secret key in a NEXT_PUBLIC variable or commit secrets to Git.

For Vercel

• Add both environment variables in Vercel's project import/settings flow.

• Select Production and Preview environments as appropriate.

• Deploy or redeploy after changing environment variables.

• In Supabase Authentication → URL Configuration, add the deployed Vercel URL to allowed redirect URLs and set the Site URL appropriately.

• Test signup, login, check-in persistence and therapist dashboard access on the deployed site.

6. Local development and deployment

Run locally

```
npm install
npm run dev
```

Open http://localhost:3000. Routes recorded during development:

• /login — sign-in

• /signup — registration

• /patient/checkin — patient check-in

• /therapist/dashboard — therapist dashboard

Deploy to Vercel

• Push the project to GitHub. Repository: https://github.com/nandana-ps/therapy-companion.

• Import the repository at https://vercel.com/new.

• Confirm the framework is detected as Next.js.

• Add the Supabase environment variables, then deploy.

• Open the generated Vercel URL and test the full flow. Deployment is complete only after the build succeeds and the live app is tested.

If a Vercel build fails, inspect the first relevant error in the build log. Fix it locally, commit and push the correction; Vercel will normally rebuild from the connected branch.

7. Current limitations and responsible use

• Follow-up questions and session-preparation content are rule-based, not generated by an integrated AI model.

• The preparation brief is for organizing a session, not a clinical assessment, diagnosis or treatment recommendation.

• The application is not an emergency or crisis-response service.

• Before using real patient information, review privacy notices, consent, retention/deletion practices, authorization and RLS policies.

• Test cross-account access: one patient must not see another patient's records; a therapist must only see assigned patients.

8. MVP test checklist

• ■ Local app starts without errors.

• ■ Signup creates a Supabase Auth user and matching profile.

• ■ Login and logout work.

• ■ Patient can complete all ten questions and save a check-in.

• ■ Check-in persists after refresh/re-login.

• ■ Therapist dashboard displays the intended patient's saved check-in.

• ■ Session preparation brief uses the correct saved record.

• ■ Unauthorized users cannot read another patient's records.

• ■ Vercel environment variables are set.

• ■ Deployed site loads and connects to Supabase.

• ■ Supabase redirect URLs include the deployed domain.

9. Repository hygiene

• Keep .env.local excluded in .gitignore.

• Never commit passwords, access tokens, service-role keys or other secrets.

• Use meaningful commits for changes.

• Keep the README aligned with what is actually implemented.

• Do not describe current rule-based features as AI-powered until an AI model/API is integrated and tested.

10. Project summary

Better, Together is a therapy continuity companion MVP connecting patient self-reported check-ins with therapist review and session preparation. It is built with Next.js, TypeScript and Supabase, with Vercel for deployment. The current value lies in structured reflection and continuity between sessions; further work should focus on secure access control, clear privacy practices, production testing and careful validation of any future AI assistance.

Repository: https://github.com/nandana-ps/therapy-companion